Lecture 3: Selection, Survivor, Crossover, and Mutation Operators in BGA

Course: Evolutionary Algorithms

Instructor: Engr. Muhammad Farooq

Learning outcomes

By the end of this lecture, students should be able to:

- explain the purpose of selection in genetic algorithms

Learning outcomes

By the end of this lecture, students should be able to:

- explain the purpose of selection in genetic algorithms
- distinguish parent selection and survivor selection

Learning outcomes

By the end of this lecture, students should be able to:

- explain the purpose of selection in genetic algorithms
- distinguish parent selection and survivor selection
- compute fitness proportionate selection probabilities

Learning outcomes

By the end of this lecture, students should be able to:

- explain the purpose of selection in genetic algorithms
- distinguish parent selection and survivor selection
- compute fitness proportionate selection probabilities
- apply roulette-wheel selection using cumulative probabilities

Learning outcomes

By the end of this lecture, students should be able to:

- explain the purpose of selection in genetic algorithms
- distinguish parent selection and survivor selection
- compute fitness proportionate selection probabilities
- apply roulette-wheel selection using cumulative probabilities
- explain stochastic remainder and stochastic universal sampling

Learning outcomes

By the end of this lecture, students should be able to:

- explain the purpose of selection in genetic algorithms
- distinguish parent selection and survivor selection
- compute fitness proportionate selection probabilities
- apply roulette-wheel selection using cumulative probabilities
- explain stochastic remainder and stochastic universal sampling
- apply tournament selection with examples

Learning outcomes

By the end of this lecture, students should be able to:

- explain the purpose of selection in genetic algorithms
- distinguish parent selection and survivor selection
- compute fitness proportionate selection probabilities
- apply roulette-wheel selection using cumulative probabilities
- explain stochastic remainder and stochastic universal sampling
- apply tournament selection with examples
- distinguish $(\mu + \lambda)$ and (μ, λ) survivor schemes

Learning outcomes

By the end of this lecture, students should be able to:

- explain the purpose of selection in genetic algorithms
- distinguish parent selection and survivor selection
- compute fitness proportionate selection probabilities
- apply roulette-wheel selection using cumulative probabilities
- explain stochastic remainder and stochastic universal sampling
- apply tournament selection with examples
- distinguish $(\mu + \lambda)$ and (μ, λ) survivor schemes
- apply uniform crossover to binary strings

Learning outcomes

By the end of this lecture, students should be able to:

- explain the purpose of selection in genetic algorithms
- distinguish parent selection and survivor selection
- compute fitness proportionate selection probabilities
- apply roulette-wheel selection using cumulative probabilities
- explain stochastic remainder and stochastic universal sampling
- apply tournament selection with examples
- distinguish $(\mu + \lambda)$ and (μ, λ) survivor schemes
- apply uniform crossover to binary strings
- explain local and global mutation for binary chromosomes

Connection with Lecture 2

In Lecture 2, we studied the BGA cycle:

1. initialize population

Connection with Lecture 2

In Lecture 2, we studied the BGA cycle:

1. initialize population
2. decode chromosomes

Connection with Lecture 2

In Lecture 2, we studied the BGA cycle:

1. initialize population
2. decode chromosomes
3. evaluate objective / fitness

Connection with Lecture 2

In Lecture 2, we studied the BGA cycle:

1. initialize population
2. decode chromosomes
3. evaluate objective / fitness
4. select parents

Connection with Lecture 2

In Lecture 2, we studied the BGA cycle:

1. initialize population
2. decode chromosomes
3. evaluate objective / fitness
4. select parents
5. apply crossover

Connection with Lecture 2

In Lecture 2, we studied the BGA cycle:

1. initialize population
2. decode chromosomes
3. evaluate objective / fitness
4. select parents
5. apply crossover
6. apply mutation

Connection with Lecture 2

In Lecture 2, we studied the BGA cycle:

1. initialize population
2. decode chromosomes
3. evaluate objective / fitness
4. select parents
5. apply crossover
6. apply mutation
7. evaluate offspring

Connection with Lecture 2

In Lecture 2, we studied the BGA cycle:

1. initialize population
2. decode chromosomes
3. evaluate objective / fitness
4. select parents
5. apply crossover
6. apply mutation
7. evaluate offspring
8. choose survivors

Connection with Lecture 2

In Lecture 2, we studied the BGA cycle:

1. initialize population
2. decode chromosomes
3. evaluate objective / fitness
4. select parents
5. apply crossover
6. apply mutation
7. evaluate offspring
8. choose survivors
9. form next generation

Connection with Lecture 2

In Lecture 2, we studied the BGA cycle:

1. initialize population
2. decode chromosomes
3. evaluate objective / fitness
4. select parents
5. apply crossover
6. apply mutation
7. evaluate offspring
8. choose survivors
9. form next generation

Lecture 3 focuses on the operators inside this cycle:

- selection operators
- survivor operators
- crossover operators and mutation operators

Selection operators

- The purpose is to identify good or above-average solutions.

Selection operators

- The purpose is to identify good or above-average solutions.
- In this process, weak solutions get fewer opportunities.

Selection operators

- The purpose is to identify good or above-average solutions.
- In this process, weak solutions get fewer opportunities.
- Good solutions may get multiple copies.

Selection operators

- The purpose is to identify good or above-average solutions.
- In this process, weak solutions get fewer opportunities.
- Good solutions may get multiple copies.
- Selection can be used before or after variation operators.

Selection operators

- The purpose is to identify good or above-average solutions.
- In this process, weak solutions get fewer opportunities.
- Good solutions may get multiple copies.
- Selection can be used before or after variation operators.

Parent selection: done before crossover/mutation, used to create the mating pool.

Survivor selection: done after offspring are generated, used to choose the next generation.

Common selection methods

Common methods are:

- Fitness proportionate selection

Common selection methods

Common methods are:

- Fitness proportionate selection
- Roulette-wheel selection

Common selection methods

Common methods are:

- Fitness proportionate selection
- Roulette-wheel selection
- Stochastic remainder roulette-wheel selection

Common selection methods

Common methods are:

- Fitness proportionate selection
- Roulette-wheel selection
- Stochastic remainder roulette-wheel selection
- Stochastic universal sampling (SUS)

Common selection methods

Common methods are:

- Fitness proportionate selection
- Roulette-wheel selection
- Stochastic remainder roulette-wheel selection
- Stochastic universal sampling (SUS)
- Tournament selection

Common selection methods

Common methods are:

- Fitness proportionate selection
- Roulette-wheel selection
- Stochastic remainder roulette-wheel selection
- Stochastic universal sampling (SUS)
- Tournament selection
- Ranking selection

Common selection methods

Common methods are:

- Fitness proportionate selection
- Roulette-wheel selection
- Stochastic remainder roulette-wheel selection
- Stochastic universal sampling (SUS)
- Tournament selection
- Ranking selection

In this lecture, we focus mainly on:

- fitness proportionate / roulette-wheel selection
- stochastic remainder selection
- SUS
- tournament selection

Fitness proportionate selection

Probability of selecting individual i from population P of size N is:

$$p_i = \frac{f_i}{\sum_{j=1}^{N} f_j}$$

- f_i is the fitness of individual i

Fitness proportionate selection

Probability of selecting individual i from population P of size N is:

$$p_i = \frac{f_i}{\sum_{j=1}^{N} f_j}$$

- f_i is the fitness of individual i
- larger fitness gives larger probability

Fitness proportionate selection

Probability of selecting individual i from population P of size N is:

$$p_i = \frac{f_i}{\sum_{j=1}^{N} f_j}$$

- f_i is the fitness of individual i
- larger fitness gives larger probability
- this method is naturally suitable for maximization problems

Fitness proportionate selection

Probability of selecting individual i from population P of size N is:

$$p_i = \frac{f_i}{\sum_{j=1}^{N} f_j}$$

- f_i is the fitness of individual i
- larger fitness gives larger probability
- this method is naturally suitable for maximization problems
- it is also known as roulette-wheel selection

Maximization vs minimization

- Fitness proportionate selection is naturally suited for **maximization**.

Maximization vs minimization

- Fitness proportionate selection is naturally suited for **maximization**.
- In maximization, larger fitness should mean larger probability.

Maximization vs minimization

- Fitness proportionate selection is naturally suited for **maximization**.
- In maximization, larger fitness should mean larger probability.
- For minimization, we usually transform the objective into a fitness value.

Maximization vs minimization

- Fitness proportionate selection is naturally suited for **maximization**.
- In maximization, larger fitness should mean larger probability.
- For minimization, we usually transform the objective into a fitness value.

One simple transformation is:

$$F_i = \frac{1}{1 + f_i}$$

So:

- use raw fitness directly for maximization
- transform objective value first for minimization

Worked example: fitness proportionate selection

Suppose we have a maximization problem with 4 individuals.

Individual	Fitness
1	10
2	20
3	30
4	40

Worked example: fitness proportionate selection

Suppose we have a maximization problem with 4 individuals.

Individual	Fitness
1	10
2	20
3	30
4	40

$$\text{Total fitness} = 10 + 20 + 30 + 40 = 100$$

Worked example: fitness proportionate selection

Suppose we have a maximization problem with 4 individuals.

Individual	Fitness
1	10
2	20
3	30
4	40

$$\text{Total fitness} = 10 + 20 + 30 + 40 = 100$$

$$p_1 = \frac{10}{100} = 0.10, \quad p_2 = \frac{20}{100} = 0.20, \quad p_3 = \frac{30}{100} = 0.30, \quad p_4 = \frac{40}{100} = 0.40$$

Roulette-wheel selection using cumulative probabilities

Individual	p_i	Cumulative P_i
1	0.10	0.10
2	0.20	0.30
3	0.30	0.60
4	0.40	1.00

Roulette-wheel selection using cumulative probabilities

Individual	p_i	Cumulative P_i
1	0.10	0.10
2	0.20	0.30
3	0.30	0.60
4	0.40	1.00

Suppose random numbers are:

$$r_1 = 0.08, \quad r_2 = 0.27, \quad r_3 = 0.73$$

Roulette-wheel selection using cumulative probabilities

Individual	p_i	Cumulative P_i
1	0.10	0.10
2	0.20	0.30
3	0.30	0.60
4	0.40	1.00

Suppose random numbers are:

$$r_1 = 0.08, \quad r_2 = 0.27, \quad r_3 = 0.73$$

Selections:

- $0.08 \in [0, 0.10) \rightarrow$ individual 1
- $0.27 \in [0.10, 0.30) \rightarrow$ individual 2
- $0.73 \in [0.60, 1.00] \rightarrow$ individual 4

Roulette-wheel selection using cumulative probabilities

Individual	p_i	Cumulative P_i
1	0.10	0.10
2	0.20	0.30
3	0.30	0.60
4	0.40	1.00

Suppose random numbers are:

$$r_1 = 0.08, \quad r_2 = 0.27, \quad r_3 = 0.73$$

Selections:

- $0.08 \in [0, 0.10) \rightarrow$ individual 1
- $0.27 \in [0.10, 0.30) \rightarrow$ individual 2
- $0.73 \in [0.60, 1.00] \rightarrow$ individual 4

Selected set: $\{1, 2, 4\}$

Why roulette-wheel selection works

- Good individuals are selected more often.

Why roulette-wheel selection works

- Good individuals are selected more often.
- Weak individuals still have some chance.

Why roulette-wheel selection works

- Good individuals are selected more often.
- Weak individuals still have some chance.
- Selection is stochastic, not fully deterministic.

Why roulette-wheel selection works

- Good individuals are selected more often.
- Weak individuals still have some chance.
- Selection is stochastic, not fully deterministic.

This helps balance:

- exploitation of good solutions
- exploration of the search space

Limitations of fitness proportionate selection

Problem 1: domination by a super-solution
Suppose fitness values are:

$$f_1 = 10, \quad f_2 = 5, \quad f_3 = 70, \quad f_4 = 7, \quad f_5 = 8$$

Limitations of fitness proportionate selection

Problem 1: domination by a super-solution
Suppose fitness values are:

$$f_1 = 10, \quad f_2 = 5, \quad f_3 = 70, \quad f_4 = 7, \quad f_5 = 8$$

$$\text{Total} = 100, \qquad p_3 = \frac{70}{100} = 0.70$$

Limitations of fitness proportionate selection

Problem 1: domination by a super-solution
Suppose fitness values are:

$$f_1 = 10, \quad f_2 = 5, \quad f_3 = 70, \quad f_4 = 7, \quad f_5 = 8$$

$$\text{Total} = 100, \qquad p_3 = \frac{70}{100} = 0.70$$

- solution 3 may dominate the population very early
- diversity may decrease too quickly
- premature convergence may happen

Limitations of fitness proportionate selection

Problem 1: domination by a super-solution
Suppose fitness values are:

$$f_1 = 10, \quad f_2 = 5, \quad f_3 = 70, \quad f_4 = 7, \quad f_5 = 8$$

$$\text{Total} = 100, \qquad p_3 = \frac{70}{100} = 0.70$$

- solution 3 may dominate the population very early
- diversity may decrease too quickly
- premature convergence may happen

Problem 2: weak pressure later

Limitations of fitness proportionate selection

Problem 1: domination by a super-solution
Suppose fitness values are:

$$f_1 = 10, \quad f_2 = 5, \quad f_3 = 70, \quad f_4 = 7, \quad f_5 = 8$$

$$\text{Total} = 100, \qquad p_3 = \frac{70}{100} = 0.70$$

- solution 3 may dominate the population very early
- diversity may decrease too quickly
- premature convergence may happen

Problem 2: weak pressure later
If fitness values become very close, then all individuals get similar probabilities, and selection becomes weak.

Remedies for roulette-wheel selection

Possible remedies:

- fitness scaling

Remedies for roulette-wheel selection

Possible remedies:

- fitness scaling
- ranking selection

Remedies for roulette-wheel selection

Possible remedies:

- fitness scaling
- ranking selection
- tournament selection

Remedies for roulette-wheel selection

Possible remedies:

- fitness scaling
- ranking selection
- tournament selection

Tournament selection is often preferred because:

- it is simple
- it is robust
- it avoids most fitness scaling problems

Stochastic remainder roulette-wheel selection

This method combines:

- deterministic selection

Stochastic remainder roulette-wheel selection

This method combines:

- deterministic selection
- probabilistic selection

Stochastic remainder roulette-wheel selection

This method combines:

- deterministic selection
- probabilistic selection

Procedure:

1. Compute Np_i for each individual

Stochastic remainder roulette-wheel selection

This method combines:

- deterministic selection
- probabilistic selection

Procedure:

1. Compute Np_i for each individual
2. Take the integer part as guaranteed copies

Stochastic remainder roulette-wheel selection

This method combines:

- deterministic selection
- probabilistic selection

Procedure:

1. Compute Np_i for each individual
2. Take the integer part as guaranteed copies
3. Use the fractional parts to fill the remaining positions probabilistically

Worked example: stochastic remainder selection

Use:

$$p_1 = 0.10, \quad p_2 = 0.20, \quad p_3 = 0.30, \quad p_4 = 0.40$$

Worked example: stochastic remainder selection

Use:

$$p_1 = 0.10, \quad p_2 = 0.20, \quad p_3 = 0.30, \quad p_4 = 0.40$$

Let population size be:

$$N = 8$$

Worked example: stochastic remainder selection

Use:

$$p_1 = 0.10, \quad p_2 = 0.20, \quad p_3 = 0.30, \quad p_4 = 0.40$$

Let population size be:

$$N = 8$$

Then:

$$Np_1 = 0.8, \quad Np_2 = 1.6, \quad Np_3 = 2.4, \quad Np_4 = 3.2$$

Worked example: stochastic remainder selection

Use:

$$p_1 = 0.10, \quad p_2 = 0.20, \quad p_3 = 0.30, \quad p_4 = 0.40$$

Let population size be:

$$N = 8$$

Then:

$$Np_1 = 0.8, \quad Np_2 = 1.6, \quad Np_3 = 2.4, \quad Np_4 = 3.2$$

Integer parts:

- individual 1 gets 0 guaranteed copies
- individual 2 gets 1 guaranteed copy
- individual 3 gets 2 guaranteed copies
- individual 4 gets 3 guaranteed copies

Worked example: stochastic remainder selection

Use:

$$p_1 = 0.10, \quad p_2 = 0.20, \quad p_3 = 0.30, \quad p_4 = 0.40$$

Let population size be:

$$N = 8$$

Then:

$$Np_1 = 0.8, \quad Np_2 = 1.6, \quad Np_3 = 2.4, \quad Np_4 = 3.2$$

Integer parts:

- individual 1 gets 0 guaranteed copies
- individual 2 gets 1 guaranteed copy
- individual 3 gets 2 guaranteed copies
- individual 4 gets 3 guaranteed copies

So we already have: $\{2, 3, 3, 4, 4, 4\}$

Filling remaining slots in stochastic remainder selection

Fractional remainders are:

$$0.8, \quad 0.6, \quad 0.4, \quad 0.2$$

Filling remaining slots in stochastic remainder selection

Fractional remainders are:

$$0.8, \quad 0.6, \quad 0.4, \quad 0.2$$

Two more positions remain to be filled.

Filling remaining slots in stochastic remainder selection

Fractional remainders are:

$$0.8, \quad 0.6, \quad 0.4, \quad 0.2$$

Two more positions remain to be filled.
A likely outcome is:

$$\{2, 3, 3, 4, 4, 4, 1, 2\}$$

Filling remaining slots in stochastic remainder selection

Fractional remainders are:

$$0.8, \quad 0.6, \quad 0.4, \quad 0.2$$

Two more positions remain to be filled.
A likely outcome is:

$$\{2, 3, 3, 4, 4, 4, 1, 2\}$$

Why is this useful?

- good solutions get their expected integer copies
- randomness is reduced
- sampling variance is smaller than plain roulette-wheel selection

Stochastic universal sampling (SUS)

SUS is another low-variance version of roulette-wheel selection.

- It uses only one random number.

Stochastic universal sampling (SUS)

SUS is another low-variance version of roulette-wheel selection.

- It uses only one random number.
- It uses equally spaced pointers.

Stochastic universal sampling (SUS)

SUS is another low-variance version of roulette-wheel selection.

- It uses only one random number.
- It uses equally spaced pointers.
- It samples the wheel more evenly.

Stochastic universal sampling (SUS)

SUS is another low-variance version of roulette-wheel selection.

- It uses only one random number.
- It uses equally spaced pointers.
- It samples the wheel more evenly.

If N individuals must be selected, use:

$$r, \quad r + \frac{1}{N}, \quad r + \frac{2}{N}, \quad \ldots, \quad r + \frac{N-1}{N}$$

Worked example: SUS

Use the same cumulative probabilities:

Individual	p_i	Cumulative P_i
1	0.10	0.10
2	0.20	0.30
3	0.30	0.60
4	0.40	1.00

Worked example: SUS

Use the same cumulative probabilities:

Individual	p_i	Cumulative P_i
1	0.10	0.10
2	0.20	0.30
3	0.30	0.60
4	0.40	1.00

Suppose we want $N = 4$ selections. Then:

$$\frac{1}{N} = \frac{1}{4} = 0.25$$

Worked example: SUS

Use the same cumulative probabilities:

Individual	p_i	Cumulative P_i
1	0.10	0.10
2	0.20	0.30
3	0.30	0.60
4	0.40	1.00

Suppose we want $N = 4$ selections. Then:

$$\frac{1}{N} = \frac{1}{4} = 0.25$$

Choose random start:

$$r = 0.12$$

Worked example: SUS

Use the same cumulative probabilities:

Individual	p_i	Cumulative P_i
1	0.10	0.10
2	0.20	0.30
3	0.30	0.60
4	0.40	1.00

Suppose we want $N = 4$ selections. Then:

$$\frac{1}{N} = \frac{1}{4} = 0.25$$

Choose random start:

$$r = 0.12$$

Pointers:

$$0.12,\ 0.37,\ 0.62,\ 0.87$$

SUS selection result

Using cumulative intervals:

- 0.12 $\rightarrow$ individual 2

SUS selection result

Using cumulative intervals:

- 0.12 $\rightarrow$ individual 2
- 0.37 $\rightarrow$ individual 3

SUS selection result

Using cumulative intervals:

- 0.12 $\rightarrow$ individual 2
- 0.37 $\rightarrow$ individual 3
- 0.62 $\rightarrow$ individual 4

SUS selection result

Using cumulative intervals:

- 0.12 $\rightarrow$ individual 2
- 0.37 $\rightarrow$ individual 3
- 0.62 $\rightarrow$ individual 4
- 0.87 $\rightarrow$ individual 4

SUS selection result

Using cumulative intervals:

- $0.12 \rightarrow$ individual 2
- $0.37 \rightarrow$ individual 3
- $0.62 \rightarrow$ individual 4
- $0.87 \rightarrow$ individual 4

Selected set: $\{2, 3, 4, 4\}$

Note: If the value exceeds 1, subtract 1 from it

SUS selection result

Using cumulative intervals:

- $0.12 \rightarrow$ individual 2
- $0.37 \rightarrow$ individual 3
- $0.62 \rightarrow$ individual 4
- $0.87 \rightarrow$ individual 4

Selected set: $\{2, 3, 4, 4\}$

SUS is useful because:

- only one random number is used
- the sampling is smoother
- variance is lower than ordinary roulette-wheel selection

Note: If the value exceeds 1, subtract 1 from it

Tournament selection

Tournament selection is one of the simplest and most widely used methods.

- randomly sample a subset of k individuals

Tournament selection

Tournament selection is one of the simplest and most widely used methods.

- randomly sample a subset of k individuals
- compare them

Tournament selection

Tournament selection is one of the simplest and most widely used methods.

- randomly sample a subset of k individuals
- compare them
- choose the best one

Tournament selection

Tournament selection is one of the simplest and most widely used methods.

- randomly sample a subset of k individuals
- compare them
- choose the best one

This is called a **tournament of size** k.

Worked example: tournament selection

Suppose $k = 4$, and the sampled subset is:

$$\{2, 6, 3, 9\}$$

Solution	Fitness
2	10.0
6	5.9
3	16.7
9	9.4

Worked example: tournament selection

Suppose $k = 4$, and the sampled subset is:

$$\{2, 6, 3, 9\}$$

Solution	Fitness
2	10.0
6	5.9
3	16.7
9	9.4

For maximization, winner is:

solution 3

Worked example: tournament selection

Suppose $k = 4$, and the sampled subset is:

$$\{2, 6, 3, 9\}$$

Solution	Fitness
2	10.0
6	5.9
3	16.7
9	9.4

For maximization, winner is:

solution 3

For minimization, winner is:

solution 6

Binary tournament selection

A common special case is:

$$k = 2$$

Binary tournament selection

A common special case is:

$$k = 2$$

This is called **binary tournament selection**.

Binary tournament selection

A common special case is:

$$k = 2$$

This is called **binary tournament selection**.
Example:

Binary tournament selection

A common special case is:

$$k = 2$$

This is called **binary tournament selection**.
Example:

Solution	Objective value
A	12
B	7

Binary tournament selection

A common special case is:

$$k = 2$$

This is called **binary tournament selection**.
Example:

Solution	Objective value
A	12
B	7

For minimization:

$$\text{winner} = B$$

Binary tournament selection

A common special case is:

$$k = 2$$

This is called **binary tournament selection**.
Example:

Solution	Objective value
A	12
B	7

For minimization:

$$\text{winner} = B$$

For maximization:

$$\text{winner} = A$$

Effect of tournament size

Tournament size controls **selection pressure**.

Small k:

- weaker pressure

Effect of tournament size

Tournament size controls **selection pressure**.
Small k:

- weaker pressure
- more diversity

Effect of tournament size

Tournament size controls **selection pressure**.

Small k:

- weaker pressure
- more diversity
- slower convergence

Effect of tournament size

Tournament size controls **selection pressure**.

Small k:

- weaker pressure
- more diversity
- slower convergence

Large k:

- stronger pressure

Effect of tournament size

Tournament size controls **selection pressure**.

Small k:

- weaker pressure
- more diversity
- slower convergence

Large k:

- stronger pressure
- faster convergence

Effect of tournament size

Tournament size controls **selection pressure**.

Small k:

- weaker pressure
- more diversity
- slower convergence

Large k:

- stronger pressure
- faster convergence
- greater risk of premature convergence

Survivor or elimination operators

After offspring are created, we must decide who survives into the next generation.

- This is called survivor selection.

Survivor or elimination operators

After offspring are created, we must decide who survives into the next generation.

- This is called survivor selection.
- It is also called elimination or environmental selection.

Survivor or elimination operators

After offspring are created, we must decide who survives into the next generation.

- This is called survivor selection.
- It is also called elimination or environmental selection.
- Two common schemes are $(\mu + \lambda)$ and (μ, λ).

Survivor or elimination operators

After offspring are created, we must decide who survives into the next generation.

- This is called survivor selection.
- It is also called elimination or environmental selection.
- Two common schemes are $(\mu + \lambda)$ and (μ, λ).

where:

$$\mu = \text{number of parents}, \qquad \lambda = \text{number of offspring}$$

$(\mu + \lambda)$ selection scheme

In this scheme:

- parent population size $= \mu$

$(\mu + \lambda)$ selection scheme

In this scheme:

- parent population size $= \mu$
- offspring population size $= \lambda$

$(\mu + \lambda)$ selection scheme

In this scheme:

- parent population size $= \mu$
- offspring population size $= \lambda$
- combine parents and offspring

$(\mu + \lambda)$ selection scheme

In this scheme:

- parent population size $= \mu$
- offspring population size $= \lambda$
- combine parents and offspring
- choose the best μ solutions from the combined population

$(\mu + \lambda)$ selection scheme

In this scheme:

- parent population size $= \mu$
- offspring population size $= \lambda$
- combine parents and offspring
- choose the best μ solutions from the combined population

So the next generation is selected from:

$$\text{parents} \cup \text{offspring}$$

Worked example: $(\mu + \lambda)$ selection

Suppose:

$$\mu = 4, \qquad \lambda = 6$$

Parent objective values

Parent	Value
P1	20
P2	12
P3	8
P4	15

Offspring objective values

Offspring	Value
O1	6
O2	10
O3	25
O4	9
O5	14
O6	7

$(\mu + \lambda)$ result

Combine all values and sort for minimization:

$$6,\ 7,\ 8,\ 9,\ 10,\ 12,\ 14,\ 15,\ 20,\ 25$$

$(\mu + \lambda)$ result

Combine all values and sort for minimization:

$$6,\ 7,\ 8,\ 9,\ 10,\ 12,\ 14,\ 15,\ 20,\ 25$$

Choose best $\mu = 4$:

$$O1,\ O6,\ P3,\ O4$$

$(\mu + \lambda)$ result

Combine all values and sort for minimization:

$$6,\ 7,\ 8,\ 9,\ 10,\ 12,\ 14,\ 15,\ 20,\ 25$$

Choose best $\mu = 4$:

$$O1,\ O6,\ P3,\ O4$$

Next generation:

- O1
- O6
- P3
- O4

Notice: parent P3 survives directly.

(μ, λ) selection scheme

In this scheme:

- parent population size $= \mu$

(μ, λ) selection scheme

In this scheme:

- parent population size $= \mu$
- offspring population size $= \lambda$, usually $\lambda > \mu$

(μ, λ) selection scheme

In this scheme:

- parent population size $= \mu$
- offspring population size $= \lambda$, usually $\lambda > \mu$
- choose the best μ solutions only from offspring

(μ, λ) selection scheme

In this scheme:

- parent population size $= \mu$
- offspring population size $= \lambda$, usually $\lambda > \mu$
- choose the best μ solutions only from offspring

Parents do **not** survive automatically.

Worked example: (μ, λ) selection

Use the same example.
Offspring values:

$$6,\ 10,\ 25,\ 9,\ 14,\ 7$$

Worked example: (μ, λ) selection

Use the same example.
Offspring values:

$$6,\ 10,\ 25,\ 9,\ 14,\ 7$$

Sort:

$$6,\ 7,\ 9,\ 10,\ 14,\ 25$$

Worked example: (μ, λ) selection

Use the same example.
Offspring values:

$$6,\ 10,\ 25,\ 9,\ 14,\ 7$$

Sort:

$$6,\ 7,\ 9,\ 10,\ 14,\ 25$$

Choose best $\mu = 4$:

$$O1,\ O6,\ O4,\ O2$$

Worked example: (μ, λ) selection

Use the same example.
Offspring values:

$$6,\ 10,\ 25,\ 9,\ 14,\ 7$$

Sort:

$$6,\ 7,\ 9,\ 10,\ 14,\ 25$$

Choose best $\mu = 4$:

$$O1,\ O6,\ O4,\ O2$$

Now parent P3 with value 8 is removed, even though it was very good.

Comparison of survivor schemes

$(\mu + \lambda)$

- good parents can survive

(μ, λ)

Comparison of survivor schemes

$(\mu + \lambda)$

- good parents can survive
- more conservative

(μ, λ)

Comparison of survivor schemes

$(\mu + \lambda)$

- good parents can survive
- more conservative
- preserves strong existing solutions

(μ, λ)

Comparison of survivor schemes

$(\mu + \lambda)$

- good parents can survive
- more conservative
- preserves strong existing solutions

(μ, λ)

- only offspring survive

Comparison of survivor schemes

$(\mu + \lambda)$

- good parents can survive
- more conservative
- preserves strong existing solutions

(μ, λ)

- only offspring survive
- more aggressive

Comparison of survivor schemes

$(\mu + \lambda)$

- good parents can survive
- more conservative
- preserves strong existing solutions

(μ, λ)

- only offspring survive
- more aggressive
- stronger renewal of population

Comparison of survivor schemes

$(\mu + \lambda)$

- good parents can survive
- more conservative
- preserves strong existing solutions

(μ, λ)

- only offspring survive
- more aggressive
- stronger renewal of population
- may lose strong parents

Crossover operators for binary variables

In Lecture 2, we studied single-point crossover.

Now we extend the discussion to:

- n-point crossover

Crossover operators for binary variables

In Lecture 2, we studied single-point crossover.

Now we extend the discussion to:

- n-point crossover
- uniform crossover

n-point crossover

In n-point crossover, we choose **n crossover points** and alternately exchange segments between two parents.

n-point crossover

In n-point crossover, we choose **n crossover points** and alternately exchange segments between two parents.
Example: let $n = 2$.

n-point crossover

In n-point crossover, we choose **n crossover points** and alternately exchange segments between two parents.
Example: let $n = 2$.
Parents:

$$P_1 = 110 \,|\, 0101 \,|\, 111$$
$$P_2 = 001 \,|\, 1010 \,|\, 000$$

n-point crossover

In n-point crossover, we choose **n crossover points** and alternately exchange segments between two parents.
Example: let $n = 2$.
Parents:

$$P_1 = 110 \,|\, 0101 \,|\, 111$$
$$P_2 = 001 \,|\, 1010 \,|\, 000$$

Crossover points are after bit 3 and after bit 7.

n-point crossover

In n-point crossover, we choose **n crossover points** and alternately exchange segments between two parents.
Example: let $n = 2$.
Parents:

$$P_1 = 110 \,|\, 0101 \,|\, 111$$
$$P_2 = 001 \,|\, 1010 \,|\, 000$$

Crossover points are after bit 3 and after bit 7.
Exchange the middle segment between the two parents.

n-point crossover

In n-point crossover, we choose **n crossover points** and alternately exchange segments between two parents.

Example: let $n = 2$.

Parents:

$$P_1 = 110 \,|\, 0101 \,|\, 111$$
$$P_2 = 001 \,|\, 1010 \,|\, 000$$

Crossover points are after bit 3 and after bit 7.

Exchange the middle segment between the two parents.

Offspring:

$$O_1 = 110 \,|\, 1010 \,|\, 111$$
$$O_2 = 001 \,|\, 0101 \,|\, 000$$

n-point crossover

In n-point crossover, we choose **n crossover points** and alternately exchange segments between two parents.

Example: let $n = 2$.

Parents:

$$P_1 = 110\,|\,0101\,|\,111$$
$$P_2 = 001\,|\,1010\,|\,000$$

Crossover points are after bit 3 and after bit 7.

Exchange the middle segment between the two parents.

Offspring:

$$O_1 = 110\,|\,1010\,|\,111$$
$$O_2 = 001\,|\,0101\,|\,000$$

- 2-point crossover divides the chromosome into 3 segments
- the middle segment is swapped, in general, alternate segments are exchanged

Uniform crossover

In uniform crossover, each bit is treated independently using a random mask.
Suppose:

$$\text{Parent } 1 = 1010111010$$

$$\text{Parent } 2 = 0101001110$$

$$\text{Mask} = 1010001110$$

Uniform crossover

In uniform crossover, each bit is treated independently using a random mask.
Suppose:

$$\text{Parent } 1 = 1010111010$$

$$\text{Parent } 2 = 0101001110$$

$$\text{Mask} = 1010001110$$

Rule:

- if mask bit $= 1$, offspring 1 takes parent 1 bit
- if mask bit $= 0$, offspring 1 takes parent 2 bit

Uniform crossover

In uniform crossover, each bit is treated independently using a random mask.
Suppose:

$$\text{Parent } 1 = 1010111010$$

$$\text{Parent } 2 = 0101001110$$

$$\text{Mask} = 1010001110$$

Rule:

- if mask bit $= 1$, offspring 1 takes parent 1 bit
- if mask bit $= 0$, offspring 1 takes parent 2 bit
- offspring 2 takes the complementary choice

Worked example: uniform crossover

Pos	P1	P2	Mask	O1	O2
1	1	0	1	1	0
2	0	1	0	1	0
3	1	0	1	1	0
4	0	1	0	1	0
5	1	0	0	0	1
6	1	0	0	0	1
7	1	1	1	1	1
8	0	1	1	0	1
9	1	1	1	1	1
10	0	0	0	0	0

Worked example: uniform crossover

Pos	P1	P2	Mask	O1	O2
1	1	0	1	1	0
2	0	1	0	1	0
3	1	0	1	1	0
4	0	1	0	1	0
5	1	0	0	0	1
6	1	0	0	0	1
7	1	1	1	1	1
8	0	1	1	0	1
9	1	1	1	1	1
10	0	0	0	0	0

Offspring 1 = 1111001010

Worked example: uniform crossover

Pos	P1	P2	Mask	O1	O2
1	1	0	1	1	0
2	0	1	0	1	0
3	1	0	1	1	0
4	0	1	0	1	0
5	1	0	0	0	1
6	1	0	0	0	1
7	1	1	1	1	1
8	0	1	1	0	1
9	1	1	1	1	1
10	0	0	0	0	0

Offspring 1 = 1111001010

Offspring 2 = 0000111110

Why uniform crossover is useful

- it gives more mixing than one-point crossover

Why uniform crossover is useful

- it gives more mixing than one-point crossover
- it treats each bit independently

Why uniform crossover is useful

- it gives more mixing than one-point crossover
- it treats each bit independently
- it explores more combinations

Why uniform crossover is useful

- it gives more mixing than one-point crossover
- it treats each bit independently
- it explores more combinations

Compared with point-based crossover:

- single-point preserves segments
- uniform crossover mixes more aggressively

Mutation operator for binary strings

Mutation introduces random changes into chromosomes.

- 0 flips to 1

Mutation operator for binary strings

Mutation introduces random changes into chromosomes.

- 0 flips to 1
- 1 flips to 0

Mutation operator for binary strings

Mutation introduces random changes into chromosomes.

- 0 flips to 1
- 1 flips to 0

Two common forms are:

- local mutation

Mutation operator for binary strings

Mutation introduces random changes into chromosomes.

- 0 flips to 1
- 1 flips to 0

Two common forms are:

- local mutation
- global mutation

Local mutation

In local mutation:

- one randomly chosen bit is flipped

Local mutation

In local mutation:

- one randomly chosen bit is flipped

Example:

1010111010

Local mutation

In local mutation:

- one randomly chosen bit is flipped

Example:

1010111010

Suppose the 4th bit is flipped.

Local mutation

In local mutation:

- one randomly chosen bit is flipped

Example:

1010111010

Suppose the 4th bit is flipped.
After mutation:

1011111010

Local mutation

In local mutation:

- one randomly chosen bit is flipped

Example:

1010111010

Suppose the 4th bit is flipped.
After mutation:

1011111010

Only one bit changes.

Global mutation

In global mutation:

- each bit is independently flipped with probability p_m

Global mutation

In global mutation:

- each bit is independently flipped with probability p_m
- this is also called per-bit mutation

Global mutation

In global mutation:

- each bit is independently flipped with probability p_m
- this is also called per-bit mutation

A common choice is:

$$p_m = \frac{1}{n}$$

where n is the chromosome length.

Worked example: global mutation

Suppose chromosome length is:

$$n = 10$$

Worked example: global mutation

Suppose chromosome length is:

$$n = 10$$

So:

$$p_m = \frac{1}{10} = 0.1$$

Worked example: global mutation

Suppose chromosome length is:

$$n = 10$$

So:

$$p_m = \frac{1}{10} = 0.1$$

Original chromosome:

$$1010111010$$

Worked example: global mutation

Suppose chromosome length is:

$$n = 10$$

So:

$$p_m = \frac{1}{10} = 0.1$$

Original chromosome:

$$1010111010$$

Suppose random numbers for each bit are:

Worked example: global mutation

Suppose chromosome length is:

$$n = 10$$

So:

$$p_m = \frac{1}{10} = 0.1$$

Original chromosome:

1010111010

Suppose random numbers for each bit are:

Bit	Random number	Flip?
1	0.05	yes
2	0.81	no
3	0.03	yes
4	0.44	no
5	0.92	no
6	0.14	no
7	0.07	yes
8	0.55	no
9	0.30	no
10	0.02	yes

Global mutation result

Flipped positions are:

$$1,\ 3,\ 7,\ 10$$

Global mutation result

Flipped positions are:

$$1,\ 3,\ 7,\ 10$$

Original chromosome:

$$1010111010$$

Global mutation result

Flipped positions are:

1, 3, 7, 10

Original chromosome:

1010111010

New chromosome:

0000110011

Global mutation result

Flipped positions are:

$$1,\ 3,\ 7,\ 10$$

Original chromosome:

$$1010111010$$

New chromosome:

$$0000110011$$

Global mutation may flip multiple bits in a chromosome.

Probability of flipping exactly k bits

If each bit flips independently with probability p_m, then:

$$P(k \text{ bits flipped}) = \binom{n}{k} p_m^k (1 - p_m)^{n-k}$$

Probability of flipping exactly k bits

If each bit flips independently with probability p_m, then:

$$P(k \text{ bits flipped}) = \binom{n}{k} p_m^k (1 - p_m)^{n-k}$$

Example:

$$n = 5, \qquad p_m = 0.1$$

Probability of flipping exactly k bits

If each bit flips independently with probability p_m, then:

$$P(k \text{ bits flipped}) = \binom{n}{k} p_m^k (1 - p_m)^{n-k}$$

Example:

$$n = 5, \qquad p_m = 0.1$$

Probability of exactly 2 flipped bits:

$$P(2) = \binom{5}{2} (0.1)^2 (0.9)^3$$

Probability of flipping exactly k bits

If each bit flips independently with probability p_m, then:

$$P(k \text{ bits flipped}) = \binom{n}{k} p_m^k (1 - p_m)^{n-k}$$

Example:

$$n = 5, \qquad p_m = 0.1$$

Probability of exactly 2 flipped bits:

$$P(2) = \binom{5}{2} (0.1)^2 (0.9)^3$$

$$= 10(0.01)(0.729) = 0.0729$$

Why mutation is important

Mutation helps by:

- introducing new genetic material

Why mutation is important

Mutation helps by:

- introducing new genetic material
- maintaining diversity

Why mutation is important

Mutation helps by:

- introducing new genetic material
- maintaining diversity
- helping escape stagnation

Why mutation is important

Mutation helps by:

- introducing new genetic material
- maintaining diversity
- helping escape stagnation
- complementing crossover

Why mutation is important

Mutation helps by:

- introducing new genetic material
- maintaining diversity
- helping escape stagnation
- complementing crossover

Without mutation:

- important alleles may be lost permanently

Why mutation is important

Mutation helps by:

- introducing new genetic material
- maintaining diversity
- helping escape stagnation
- complementing crossover

Without mutation:

- important alleles may be lost permanently

Why mutation is important

Mutation helps by:

- introducing new genetic material
- maintaining diversity
- helping escape stagnation
- complementing crossover

Without mutation:

- important alleles may be lost permanently

With too much mutation:

- search becomes random

Relationship between operators

- **Selection** chooses promising individuals.

Relationship between operators

- **Selection** chooses promising individuals.
- **Crossover** recombines useful information.

Relationship between operators

- **Selection** chooses promising individuals.
- **Crossover** recombines useful information.
- **Mutation** injects new variation.

Relationship between operators

- **Selection** chooses promising individuals.
- **Crossover** recombines useful information.
- **Mutation** injects new variation.
- **Survivor selection** preserves better individuals into the next generation.

Relationship between operators

- **Selection** chooses promising individuals.
- **Crossover** recombines useful information.
- **Mutation** injects new variation.
- **Survivor selection** preserves better individuals into the next generation.

Together, these operators balance:

- exploitation of good solutions
- exploration of new solutions

Comparison table

Operator	Main idea	Strength	Limitation
Fitness proportionate	probability proportional to fitness	intuitive	scaling problems
Stochastic remainder	integer copies + probabilistic remainder	lower variance	slightly more complex
SUS	evenly spaced sampling	stable sampling	still depends on scaling
Tournament	best in random subset	simple and robust	depends on k
$(\mu + \lambda)$	best from parents + offspring	preserves strong parents	can reduce renewal
(μ, λ)	best from offspring only	stronger renewal	may lose strong parents
Uniform crossover	bitwise mixing using mask	strong recombination	may break building blocks
Global mutation	per-bit flipping	maintains diversity	too much becomes noisy

Quick quiz

1. Fitness proportionate selection is naturally suited for:

- A. minimization only

Quick quiz

1. Fitness proportionate selection is naturally suited for:

- A. minimization only
- B. maximization

Quick quiz

1. Fitness proportionate selection is naturally suited for:

- A. minimization only
- B. maximization
- C. survivor selection only

Quick quiz

1. Fitness proportionate selection is naturally suited for:

- A. minimization only
- B. maximization
- C. survivor selection only
- D. no optimization problem

Quick quiz

1. Fitness proportionate selection is naturally suited for:

- A. minimization only
- B. maximization
- C. survivor selection only
- D. no optimization problem

Answer: B

Quick quiz

1. Fitness proportionate selection is naturally suited for:

- A. minimization only
- B. maximization
- C. survivor selection only
- D. no optimization problem

Answer: B

2. In binary tournament selection, tournament size is:

- A. 1
- B. 2

Quick quiz

1. Fitness proportionate selection is naturally suited for:

- A. minimization only
- B. maximization
- C. survivor selection only
- D. no optimization problem

Answer: B

2. In binary tournament selection, tournament size is:

- A. 1
- B. 2
- C. 3

Quick quiz

1. Fitness proportionate selection is naturally suited for:

- A. minimization only
- B. maximization
- C. survivor selection only
- D. no optimization problem

Answer: B

2. In binary tournament selection, tournament size is:

- A. 1
- B. 2
- C. 3
- D. population size

Quick quiz

1. Fitness proportionate selection is naturally suited for:

- A. minimization only
- B. maximization
- C. survivor selection only
- D. no optimization problem

Answer: B

2. In binary tournament selection, tournament size is:

- A. 1
- B. 2
- C. 3
- D. population size

Quick quiz

1. Fitness proportionate selection is naturally suited for:

- A. minimization only
- B. maximization
- C. survivor selection only
- D. no optimization problem

Answer: B

2. In binary tournament selection, tournament size is:

- A. 1
- B. 2
- C. 3
- D. population size

Quick quiz

1. Fitness proportionate selection is naturally suited for:

- A. minimization only
- B. maximization
- C. survivor selection only
- D. no optimization problem

Answer: B

2. In binary tournament selection, tournament size is:

- A. 1
- B. 2
- C. 3
- D. population size

Answer: B

Quick quiz

3. In (μ, λ) selection, the next generation is chosen from:

- A. parents only

Quick quiz

3. In (μ, λ) selection, the next generation is chosen from:

- A. parents only
- B. parents and offspring

Quick quiz

3. In (μ, λ) selection, the next generation is chosen from:

- A. parents only
- B. parents and offspring
- C. offspring only

Quick quiz

3. In (μ, λ) selection, the next generation is chosen from:

- A. parents only
- B. parents and offspring
- C. offspring only
- D. the worst individuals

Quick quiz

3. In (μ, λ) selection, the next generation is chosen from:

- A. parents only
- B. parents and offspring
- C. offspring only
- D. the worst individuals

Answer: C

Quick quiz

3. In (μ, λ) selection, the next generation is chosen from:

- A. parents only
- B. parents and offspring
- C. offspring only
- D. the worst individuals

Answer: C

4. Uniform crossover uses:

- A. one crossover point only
- B. two crossover points only

Quick quiz

3. In (μ, λ) selection, the next generation is chosen from:

- A. parents only
- B. parents and offspring
- C. offspring only
- D. the worst individuals

Answer: C

4. Uniform crossover uses:

- A. one crossover point only
- B. two crossover points only
- C. a bit mask

Quick quiz

3. In (μ, λ) selection, the next generation is chosen from:

- A. parents only
- B. parents and offspring
- C. offspring only
- D. the worst individuals

Answer: C

4. Uniform crossover uses:

- A. one crossover point only
- B. two crossover points only
- C. a bit mask
- D. only mutation probability

Quick quiz

3. In (μ, λ) selection, the next generation is chosen from:

- A. parents only
- B. parents and offspring
- C. offspring only
- D. the worst individuals

Answer: C

4. Uniform crossover uses:

- A. one crossover point only
- B. two crossover points only
- C. a bit mask
- D. only mutation probability

Quick quiz

3. In (μ, λ) selection, the next generation is chosen from:

- A. parents only
- B. parents and offspring
- C. offspring only
- D. the worst individuals

Answer: C

4. Uniform crossover uses:

- A. one crossover point only
- B. two crossover points only
- C. a bit mask
- D. only mutation probability

Quick quiz

3. In (μ, λ) selection, the next generation is chosen from:

- A. parents only
- B. parents and offspring
- C. offspring only
- D. the worst individuals

Answer: C

4. Uniform crossover uses:

- A. one crossover point only
- B. two crossover points only
- C. a bit mask
- D. only mutation probability

Answer: C

End-of-lecture summary

Students should leave this lecture knowing:

- parent selection and survivor selection are different

End-of-lecture summary

Students should leave this lecture knowing:

- parent selection and survivor selection are different
- roulette-wheel selection uses cumulative probabilities

End-of-lecture summary

Students should leave this lecture knowing:

- parent selection and survivor selection are different
- roulette-wheel selection uses cumulative probabilities
- stochastic remainder and SUS reduce sampling noise

End-of-lecture summary

Students should leave this lecture knowing:

- parent selection and survivor selection are different
- roulette-wheel selection uses cumulative probabilities
- stochastic remainder and SUS reduce sampling noise
- tournament selection is simple and robust

End-of-lecture summary

Students should leave this lecture knowing:

- parent selection and survivor selection are different
- roulette-wheel selection uses cumulative probabilities
- stochastic remainder and SUS reduce sampling noise
- tournament selection is simple and robust
- $(\mu + \lambda)$ and (μ, λ) create different survivor behavior

End-of-lecture summary

Students should leave this lecture knowing:

- parent selection and survivor selection are different
- roulette-wheel selection uses cumulative probabilities
- stochastic remainder and SUS reduce sampling noise
- tournament selection is simple and robust
- $(\mu + \lambda)$ and (μ, λ) create different survivor behavior
- uniform crossover mixes binary strings bit by bit

End-of-lecture summary

Students should leave this lecture knowing:

- parent selection and survivor selection are different
- roulette-wheel selection uses cumulative probabilities
- stochastic remainder and SUS reduce sampling noise
- tournament selection is simple and robust
- $(\mu + \lambda)$ and (μ, λ) create different survivor behavior
- uniform crossover mixes binary strings bit by bit
- mutation flips bits and maintains diversity

Transition to next lecture

Lecture 4: Real-Coded Genetic Algorithm

In Lecture 2 and Lecture 3, we studied binary-coded GA and its operators.
In the next lecture, we will solve optimization problems directly using real-valued chromosomes.